

PROGRAMME D'INTRODUCTION
À L'INTELLIGENCE ARTIFICIELLE (PIIA)

Spécial Machine Learning • 2^e Cohorte • Promotion 2026

MATHÉMATIQUES FONDAMENTALES POUR IA

*Calcul numérique | Algèbre linéaire | Probabilités
Statistiques | Mathématiques discrètes*

*« Si les gens ne croient pas que les mathématiques sont
simples, c'est uniquement parce qu'ils ne réalisent pas à quel point la
vie est compliquée. »*

— John von Neumann —

Liste des notations

Notation	Définition
Algèbre et Espaces	
$\mathbb{R}$	Ensemble des nombres réels
$\mathbb{R}^n$	Espace vectoriel réel à n dimensions
$\mathbf{x}, \mathbf{y}$	Vecteurs (écrits en minuscules grasses)
A, X, W	Matrices (écrites en majuscules)
$A^\top$	Transposée de la matrice A
A^{-1}	Inverse de la matrice A
$\text{tr}(A)$	Trace de la matrice A
$\text{rg}(A)$	Rang de la matrice A
$\ \mathbf{x}\ $	Norme euclidienne (ou norme L_2) d'un vecteur
$\kappa(A)$	Nombre de conditionnement d'une matrice A
Calcul Différentiel et Optimisation	
$\nabla f(\mathbf{x})$	Gradient de la fonction scalaire f évalué au point $\mathbf{x}$
$\nabla^2 f(\mathbf{x})$	Matrice hessienne de la fonction f évaluée au point $\mathbf{x}$
$J_f(\mathbf{x})$	Matrice jacobienne de la fonction vectorielle f
$\frac{\partial f}{\partial x_i}$	Dérivée partielle de f par rapport à la variable x_i
$\mathcal{L}(\theta)$	Fonction de coût (ou de perte) paramétrée par θ
α	Taux d'apprentissage (<i>learning rate</i>)

Liste des sigles et abréviations

Sigle	Signification
IA	Intelligence Artificielle
ML	Machine Learning (Apprentissage Automatique)
DL	Deep Learning (Apprentissage Profond)
ACP	Analyse en Composantes Principales
SVD	Singular Value Decomposition (Décomposition en Valeurs Singulières)
RNN	Recurrent Neural Network (Réseau de Neurones Récurrent)
LSTM	Long Short-Term Memory (Mémoire à Long Court Terme)
LLM	Large Language Model (Grand Modèle de Langage)
MSE	Mean Squared Error (Erreur Quadratique Moyenne)
GPU	Graphics Processing Unit (Processeur Graphique)
LoRA	Low-Rank Adaptation (Adaptation de Rang Faible)

Table des figures

1.1	La fonction sigmoïde et sa dérivée. On observe que la dérivée est maximale en $x = 0$ (valeur 0.25) et tend rapidement vers 0 pour $ x $ grand. C'est la cause du problème d'évanouissement du gradient dans les réseaux profonds.	4
1.2	Lignes de niveau d'une fonction de coût convexe quadratique. Les vecteurs rouges représentent le gradient ∇f . On remarque géométriquement qu'ils sont toujours orthogonaux aux lignes de niveau et pointent vers l'extérieur (direction de plus forte croissance).	7
1.3	Un point-selle (ou col) en $(0,0)$. Le gradient y est nul (le terrain est localement plat), mais ce n'est ni un minimum ni un maximum. L'analyse des valeurs propres de la matrice hessienne est indispensable pour identifier cette géométrie (ici, une valeur propre positive et une négative).	11
1.4	Trajectoire des poids lors de la descente de gradient. Si le pas d'apprentissage α est bien calibré (vert), l'algorithme converge efficacement. S'il est trop grand (rouge), l'algorithme "rebondit" sur les parois de la vallée d'erreur (oscillations), ce qui ralentit considérablement la convergence, voire provoque une divergence.	13
2.1	Interprétation géométrique du produit matriciel. Une matrice $A \in \mathbb{R}^{2 \times 2}$ appliquée à une grille de points dans $\mathbb{R}^2$ agit comme une déformation de l'espace (ici une combinaison de rotation et d'étirement). L'aire d'un carré unitaire formé par les vecteurs de base devient un parallélogramme dont la surface est exactement égale au déterminant de A	19
2.2	Application du Théorème Spectral à une matrice de covariance. Le nuage de points génère une matrice symétrique définie positive. La diagonalisation révèle les axes naturels de la distribution : les vecteurs propres. La longueur de l'axe principal (en rouge) est proportionnelle à la racine de la valeur propre associée λ_1 , justifiant mathématiquement le principe de l'Analyse en Composantes Principales (ACP).	23
2.3	Illustration du Théorème d'Eckart-Young via la Décomposition en Valeurs Singulières (SVD). Une matrice (ici représentée par une carte de chaleur de 100×100) de rang plein peut être approchée avec une fidélité étonnante en ne conservant qu'une fraction de ses valeurs singulières. Avec un rang $k = 20$, 99% de la variance structurelle est préservée pour un stockage drastiquement réduit.	24
2.4	Impact du conditionnement matriciel κ sur l'optimisation. À gauche : la matrice Hessienne est l'identité ($\kappa = 1$), les lignes de niveau sont parfaitement circulaires et l'algorithme converge en ligne droite. À droite : $\kappa = 15$ crée un "ravin". Le gradient (qui est orthogonal aux lignes de niveau elliptiques) pointe presque exclusivement vers les murs et très peu vers le minimum. L'algorithme rebondit violemment.	27

Liste des tableaux

1.1	Dérivées usuelles, dont deux fonctions d'activation classiques en apprentissage profond (sigmoïde, tangente hyperbolique).	3
-----	--	---

Table des matières

Liste des notations	ii
Liste des sigles et abréviations	iii
Table des figures	iv
Liste des tableaux	v
1 Calcul numérique	1
Introduction générale	1
1.1 Préliminaires : Notion de fonction en Machine Learning	2
1.2 Dérivées d'une fonction d'une variable	2
1.3 Fonctions de plusieurs variables : Le monde réel de l'IA	4
1.4 Dérivées partielles de fonctions à plusieurs variables	4
1.5 Règle de la chaîne et différentiation composée	5
1.6 Gradient d'une fonction scalaire	6
1.7 Matrice jacobienne et matrice hessienne	8
1.8 Applications à l'optimisation	10
1.8.1 Points critiques et condition nécessaire d'optimalité	10
1.8.2 Convexité et caractérisation par la hessienne	11
1.8.3 Algorithme de descente de gradient	12
Points clés du chapitre	14
Exercices et Travaux Pratiques	15
2 Algèbre linéaire	17
Introduction générale	17
2.1 Scalaires, vecteurs, matrices, tenseurs et dimensionnalité	18
2.2 Produit matriciel, Trace, Rang et Compression Sémantique	19
2.3 Inversion Matricielle, Équations Normales et Conditionnement	21
2.4 Valeurs propres, Vecteurs propres et Dynamique des RNN	22
2.5 SVD : Décomposition en Valeurs Singulières et Espaces Latents	23
Points clés du chapitre	25
Exercices et Travaux Pratiques	26

Chapitre 1

Calcul numérique

Introduction générale

Le calcul différentiel constitue le socle analytique sur lequel repose la quasi-totalité des méthodes modernes d'apprentissage automatique (*Machine Learning*) et d'apprentissage profond (*Deep Learning*). Entraîner un modèle revient presque toujours à résoudre un problème d'optimisation : trouver les paramètres qui minimisent une fonction de coût mesurant l'écart entre les prédictions et les données réelles. Or, pour y parvenir, il faut savoir dans quelle direction et de combien cette fonction varie lorsqu'on modifie légèrement ses paramètres : c'est précisément le rôle du calcul différentiel.

Ce chapitre construit progressivement les outils nécessaires pour généraliser ces concepts aux fonctions à de très nombreuses variables, comme les poids d'un réseau de neurones. Après un rappel des règles de dérivation fondamentales, nous introduirons les trois objets centraux du chapitre : le **gradient**, qui indique la direction de plus forte variation ; la **matrice jacobienne**, qui le généralise aux fonctions vectorielles ; et la **matrice hessienne**, qui capture l'information de courbure (nécessaire pour identifier un minimum ou un point-selle).

La **règle de la chaîne** y joue un rôle particulier : appliquée systématiquement à la structure en couches d'un réseau de neurones, elle donne naissance à l'algorithme de *rétropropagation du gradient* (*backpropagation*). Le chapitre se conclut par l'application directe de ces outils avec la **descente de gradient**, brique algorithmique fondamentale de l'IA moderne.

Objectif

Objectif : maîtriser les outils du calcul différentiel nécessaires à l'optimisation des modèles de *Machine Learning*.

Masse horaire : 4 h de cours • 3 h de TD/TP.

• Outils & bibliothèques

Python, NumPy, SymPy

1.1 Préliminaires : Notion de fonction en Machine Learning

Avant d'aborder le calcul de dérivées, il est crucial de rappeler ce qu'est une fonction, car en Machine Learning, absolument tout est modélisé sous forme de fonctions.

Définition 1.1: Fonction

Une fonction f est une relation qui associe à chaque élément x d'un ensemble de départ E (le domaine) un unique élément y d'un ensemble d'arrivée F (le codomaine). On note :

$$f: E \rightarrow F, \quad x \mapsto f(x) = y$$

Interprétation en IA : Dans le contexte de l'Intelligence Artificielle, on rencontre principalement deux grands types de fonctions :

- **Le Modèle (Prédiction) :** Une fonction $f_\theta(x) = \hat{y}$ qui prend en entrée des données x (ex : les pixels d'une image) et retourne une prédiction $\hat{y}$ (ex : la probabilité que ce soit un chat). Cette fonction dépend de paramètres internes θ (les poids).
- **La Fonction de Coût (Optimisation) :** Une fonction $\mathcal{L}(\theta)$ qui évalue à quel point le modèle se trompe. Son ensemble de départ est l'espace des paramètres (souvent immense, $\mathbb{R}^n$), et son ensemble d'arrivée est un simple réel ($\mathbb{R}$). **L'objectif de l'apprentissage est de trouver le θ qui rend $\mathcal{L}(\theta)$ le plus petit possible.**

1.2 Dérivées d'une fonction d'une variable

Pour minimiser notre fonction de coût $\mathcal{L}$, nous devons comprendre comment elle varie. Commençons par le cas le plus simple : une fonction f qui ne dépend que d'une seule variable réelle x .

Définition 1.2: Dérivée en un point

Soit $f: I \rightarrow \mathbb{R}$ une fonction définie sur un intervalle ouvert $I \subset \mathbb{R}$, et soit $x_0 \in I$. On dit que f est **dérivable** en x_0 si la limite

$$f'(x_0) = \lim_{h \rightarrow 0} \frac{f(x_0 + h) - f(x_0)}{h}$$

existe et est finie. Le réel $f'(x_0)$ est alors appelé **nombre dérivé** de f en x_0 .

Remarque 1.1: Interprétation

Géométriquement, $f'(x_0)$ est la pente de la tangente à la courbe représentative de f au point $(x_0, f(x_0))$. En apprentissage automatique, $f'(x_0)$ mesure la *sensibilité* de f à une petite variation de son entrée au voisinage de x_0 — une notion centrale pour comprendre comment ajuster les paramètres d'un modèle. Si $f'(x_0)$ est grand (en valeur absolue), modifier légèrement x_0 modifiera fortement la sortie de f .

Propriété 1.1: Règles de dérivation

Soient $f, g: I \rightarrow \mathbb{R}$ deux fonctions dérivables en $x \in I$ et $\lambda \in \mathbb{R}$. Alors :

$$\begin{aligned}
 (\text{Linéarité}) \quad & (\lambda f + g)'(x) = \lambda f'(x) + g'(x), \\
 (\text{Produit}) \quad & (fg)'(x) = f'(x)g(x) + f(x)g'(x), \\
 (\text{Quotient}) \quad & \left(\frac{f}{g}\right)'(x) = \frac{f'(x)g(x) - f(x)g'(x)}{g(x)^2}, \quad g(x) \neq 0, \\
 (\text{Composée}) \quad & (g \circ f)'(x) = g'(f(x)) f'(x).
 \end{aligned}$$

Fonction	Dérivée
$f(x) = x^n, n \in \mathbb{R}$	$f'(x) = n x^{n-1}$
$f(x) = e^x$	$f'(x) = e^x$
$f(x) = \ln(x)$	$f'(x) = \frac{1}{x}$
$f(x) = \sigma(x) = \frac{1}{1 + e^{-x}}$	$f'(x) = \sigma(x)(1 - \sigma(x))$
$f(x) = \tanh(x)$	$f'(x) = 1 - \tanh^2(x)$

TABLE 1.1 – Dérivées usuelles, dont deux fonctions d'activation classiques en apprentissage profond (sigmoïde, tangente hyperbolique).

Exemple 1.1: Dérivée de la fonction sigmoïde

La fonction sigmoïde $\sigma(x) = (1 + e^{-x})^{-1}$ s'écrit comme composée $g \circ f$ avec $f(x) = 1 + e^{-x}$ et $g(u) = u^{-1}$. La règle de la fonction composée donne

$$\sigma'(x) = -(1 + e^{-x})^{-2} \cdot (-e^{-x}) = \frac{e^{-x}}{(1 + e^{-x})^2} = \sigma(x)(1 - \sigma(x)),$$

en utilisant $1 - \sigma(x) = \frac{e^{-x}}{1 + e^{-x}}$. Cette identité est ce qui rend la sigmoïde particulièrement économique à différencier lors de la rétropropagation : on n'a pas besoin de recalculer des exponentielles, on réutilise simplement la valeur de la fonction elle-même !

Application :

Soit la fonction de perte locale typique pour un neurone linéaire avec cible $y = 3$: $\mathcal{L}(w) = \frac{1}{2}(2w - 3)^2$. Calculez $\mathcal{L}'(w)$.

Correction :

Ici, $\mathcal{L}$ est une fonction composée. Posons $u(w) = 2w - 3$. Alors $\mathcal{L}(w) = \frac{1}{2}u(w)^2$.

D'après la règle de la chaîne pour une variable : $\mathcal{L}'(w) = \frac{1}{2} \times 2u(w) \times u'(w)$.

Comme $u'(w) = 2$, on a : $\mathcal{L}'(w) = (2w - 3) \times 2 = 4w - 6$.

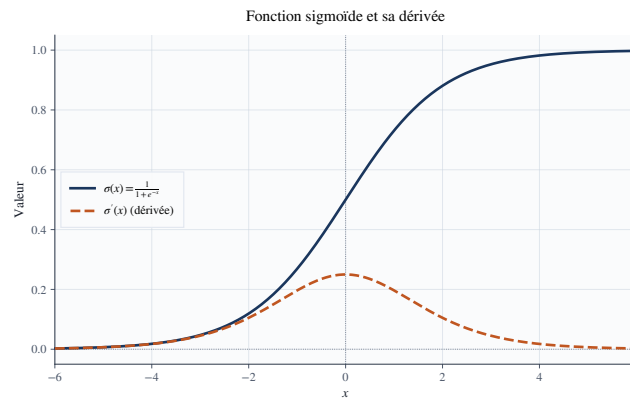


FIGURE 1.1 – La fonction sigmoïde et sa dérivée. On observe que la dérivée est maximale en $x = 0$ (valeur 0.25) et tend rapidement vers 0 pour $|x|$ grand. C'est la cause du problème d'évanouissement du gradient dans les réseaux profonds.

1.3 Fonctions de plusieurs variables : Le monde réel de l'IA

Dans la section précédente, nous avons étudié $f: \mathbb{R} \rightarrow \mathbb{R}$. Mais un modèle de Machine Learning possède rarement un seul paramètre. Un simple réseau de neurones peut en posséder des millions.

Avant de parler de dérivées partielles, il faut comprendre les **fonctions à plusieurs variables**. Ce sont des fonctions du type :

$$f: \mathbb{R}^n \rightarrow \mathbb{R}, \quad \mathbf{x} = (x_1, x_2, \dots, x_n) \mapsto f(x_1, x_2, \dots, x_n)$$

L'entrée est un vecteur (une liste de nombres), et la sortie est un scalaire (un nombre unique, par exemple l'erreur globale du modèle).

Représentation et Lignes de niveau : Pour $n = 2$ (deux paramètres, par exemple le poids w et le biais b), on peut visualiser $f(w, b)$ comme une surface en 3D (une montagne). Les "lignes de niveau" (*contour plots*) relient les points (w, b) pour lesquels la fonction vaut la même chose (l'erreur est constante). L'optimisation consiste à trouver le fond de la vallée.

1.4 Dérivées partielles de fonctions à plusieurs variables

Puisque nous sommes sur une surface multidimensionnelle, la notion de "pente" dépend de la direction dans laquelle on regarde. C'est l'essence de la dérivée partielle.

Définition 1.3: Dérivée partielle

Soit $f: U \rightarrow \mathbb{R}$ définie sur un ouvert $U \subset \mathbb{R}^n$, et soit $\mathbf{x} = (x_1, \dots, x_n) \in U$. La **dérivée partielle** de f par rapport à x_i en $\mathbf{x}$ est

$$\frac{\partial f}{\partial x_i}(\mathbf{x}) = \lim_{h \rightarrow 0} \frac{f(x_1, \dots, x_i + h, \dots, x_n) - f(\mathbf{x})}{h},$$

lorsque cette limite existe. Elle mesure le taux de variation de f lorsque x_i varie, les autres coordonnées étant maintenues fixes.

Interprétation pratique : En IA, si f est notre fonction de coût, $\frac{\partial f}{\partial w_1}$ nous dit : "Si je gèle tous les poids de mon réseau sauf w_1 , de combien mon erreur va-t-elle monter ou descendre si je modifie un tout petit peu w_1 ?".

Exemple 1.2: Calcul de dérivées partielles

Soit $f(x, y) = x^2y + \sin(xy)$. Alors

$$\frac{\partial f}{\partial x}(x, y) = 2xy + y \cos(xy), \quad \frac{\partial f}{\partial y}(x, y) = x^2 + x \cos(xy).$$

Explication : Pour trouver $\frac{\partial f}{\partial x}$, on considère y comme une constante stricte (comme si c'était le nombre 5). La dérivée de $x^2 \times (\text{constante})$ est $2x \times (\text{constante})$. La dérivée de $\sin(x \times \text{constante})$ est $\text{constante} \times \cos(x \times \text{constante})$ (règle de la chaîne). D'où le résultat. On fait l'inverse pour $\frac{\partial f}{\partial y}$ en figeant x .

Exercice d'application (Erreur quadratique) :

Soit la fonction d'erreur d'un modèle de régression linéaire pour un point de donnée ($x = 2, y = 5$) : $\mathcal{L}(w, b) = (2w + b - 5)^2$. Calculez $\frac{\partial \mathcal{L}}{\partial w}$ et $\frac{\partial \mathcal{L}}{\partial b}$.

Correction :

Posons $u(w, b) = 2w + b - 5$. Ainsi $\mathcal{L}(w, b) = u(w, b)^2$.

1. Pour $\frac{\partial \mathcal{L}}{\partial w}$, b est constant. $\frac{\partial u}{\partial w} = 2$.

Donc $\frac{\partial \mathcal{L}}{\partial w} = 2 \times u(w, b) \times \frac{\partial u}{\partial w} = 2(2w + b - 5) \times 2 = 4(2w + b - 5)$.

2. Pour $\frac{\partial \mathcal{L}}{\partial b}$, w est constant. $\frac{\partial u}{\partial b} = 1$.

Donc $\frac{\partial \mathcal{L}}{\partial b} = 2 \times u(w, b) \times \frac{\partial u}{\partial b} = 2(2w + b - 5) \times 1 = 2(2w + b - 5)$.

Remarque 1.2: Vers le calcul symbolique

Sous SymPy, ces calculs se vérifient avec `sympy.diff(f, x)` et `sympy.diff(f, y)`. En pratique, pour les fonctions de coût utilisées en Machine Learning, ces calculs sont automatisés par *différentiation automatique* (Autograd dans PyTorch ou GradientTape dans TensorFlow) plutôt que par dérivation symbolique mais la compréhension du calcul manuel reste indispensable pour interpréter les résultats et déboguer les modèles.

1.5 Règle de la chaîne et différentiation composée

L'architecture d'un réseau de neurones (Deep Learning) est fondamentalement une composition géante de fonctions mathématiques. Une couche est imbriquée dans une autre : $f(x) = f_3(f_2(f_1(x)))$. Pour dériver cela, nous avons impérativement besoin de la version vectorielle de la règle de la chaîne.

Théorème 1.1: Règle de la chaîne (cas vectoriel)

Soit $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ différentiable en $\mathbf{x}$ et $g: \mathbb{R}^m \rightarrow \mathbb{R}^p$ différentiable en $f(\mathbf{x})$. Alors $h = g \circ f$ est différentiable en $\mathbf{x}$ et sa matrice jacobienne vérifie

$$J_h(\mathbf{x}) = J_g(f(\mathbf{x})) J_f(\mathbf{x}),$$

où le produit du membre de droite est un produit matriciel.

Démonstration. Par différentiabilité de g en $f(\mathbf{x})$ et de f en $\mathbf{x}$, on a les développements au premier ordre

$$f(\mathbf{x} + \mathbf{h}) = f(\mathbf{x}) + J_f(\mathbf{x})\mathbf{h} + o(\|\mathbf{h}\|), \quad g(\mathbf{y} + \mathbf{k}) = g(\mathbf{y}) + J_g(\mathbf{y})\mathbf{k} + o(\|\mathbf{k}\|).$$

En posant $\mathbf{y} = f(\mathbf{x})$ et $\mathbf{k} = J_f(\mathbf{x})\mathbf{h} + o(\|\mathbf{h}\|)$, la composition donne

$$h(\mathbf{x} + \mathbf{h}) = g(\mathbf{y}) + J_g(\mathbf{y})J_f(\mathbf{x})\mathbf{h} + o(\|\mathbf{h}\|),$$

ce qui identifie $J_h(\mathbf{x}) = J_g(f(\mathbf{x}))J_f(\mathbf{x})$. ■

Remarque 1.3: Lien avec la rétropropagation

La règle de la chaîne est le principe mathématique fondateur de l'algorithme de **rétropropagation du gradient** : un réseau de neurones étant une composition de fonctions (couches), le gradient de la fonction de coût par rapport aux paramètres de chaque couche s'obtient en multipliant les jacobiniennes locales, de la sortie (l'erreur finale) vers l'entrée. C'est l'application du concept de *graphe de calcul*, où l'on fait circuler les dérivées à rebours via le produit matriciel stipulé par ce théorème.

Exemple 1.3: Composition scalaire

Soit $z = f(y)$ avec $y = g(x)$, $f(y) = y^2$ et $g(x) = \sin(x)$. Alors

$$\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx} = 2y \cdot \cos(x) = 2\sin(x) \cos(x) = \sin(2x).$$

Ici, on voit l'élégance de la notation de Leibniz : "les dy s'annulent virtuellement".

1.6 Gradient d'une fonction scalaire

Regroupons maintenant nos dérivées partielles dans un objet mathématique unique et surpuissant.

Définition 1.4: Gradient

Soit $f: U \rightarrow \mathbb{R}$ différentiable sur un ouvert $U \subset \mathbb{R}^n$. Le **gradient** de f en $\mathbf{x} \in U$ est le vecteur des dérivées partielles

$$\nabla f(\mathbf{x}) = \begin{pmatrix} \frac{\partial f}{\partial x_1}(\mathbf{x}) \\ \vdots \\ \frac{\partial f}{\partial x_n}(\mathbf{x}) \end{pmatrix} \in \mathbb{R}^n.$$

Propriété 1.2: Interprétation géométrique du gradient

En tout point $\mathbf{x}$ où $\nabla f(\mathbf{x}) \neq \mathbf{0}$:

1. $\nabla f(\mathbf{x})$ pointe dans la direction de plus forte croissance de f ; $-\nabla f(\mathbf{x})$ pointe dans la direction de plus forte *décroissance*;
2. $\nabla f(\mathbf{x})$ est orthogonal à la ligne (ou surface) de niveau de f passant par $\mathbf{x}$;
3. pour toute direction unitaire $\mathbf{u} \in \mathbb{R}^n$, la dérivée directionnelle de f en $\mathbf{x}$ dans la direction $\mathbf{u}$ vaut $\nabla f(\mathbf{x}) \cdot \mathbf{u}$, maximale lorsque $\mathbf{u}$ est colinéaire et de même sens que $\nabla f(\mathbf{x})$.

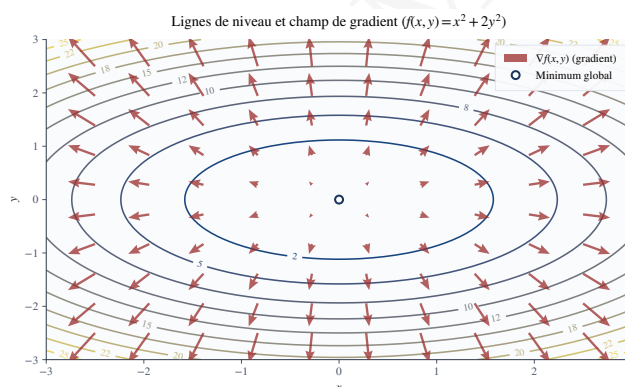


FIGURE 1.2 – Lignes de niveau d’une fonction de coût convexe quadratique. Les vecteurs rouges représentent le gradient ∇f . On remarque géométriquement qu’ils sont toujours orthogonaux aux lignes de niveau et pointent vers l’extérieur (direction de plus forte croissance).

Remarque 1.4: Pourquoi la descente de gradient fonctionne

C’est précisément le point (1) qui justifie l’algorithme de descente de gradient. Si vous êtes un aveugle cherchant le bas d’une vallée (minimiser l’erreur), la meilleure tactique locale est de tâter le sol autour de vous et de faire un pas dans la direction qui descend le plus vite : c’est la direction de $-\nabla f(\mathbf{x})$.

Exemple :

Soit $f(x, y) = x^2 + y^2$ (un bol parabolique dont le minimum est en $(0, 0)$). Calculons le gradient au point $A = (2, 1)$.

Correction :

Les dérivées partielles sont $\frac{\partial f}{\partial x} = 2x$ et $\frac{\partial f}{\partial y} = 2y$.

Donc $\nabla f(x, y) = \begin{pmatrix} 2x \\ 2y \end{pmatrix}$. Au point $A(2, 1)$, le gradient est $\nabla f(2, 1) = \begin{pmatrix} 4 \\ 2 \end{pmatrix}$.

Cela signifie que pour faire monter f le plus vite possible depuis le point $(2, 1)$, il faut avancer de 4 unités selon x pour chaque 2 unités selon y . Pour **minimiser** f (ce que l'on veut en ML), on fera un pas dans la direction *opposée* : $-\begin{pmatrix} 4 \\ 2 \end{pmatrix}$, ce qui pointe bel et bien vers l'origine $(0, 0)$!

1.7 Matrice jacobienne et matrice hessienne

Si le gradient est la "dérivée" d'une fonction scalaire, comment généraliser cela à une fonction vectorielle (qui sort un vecteur, comme une couche réseau de neurones)? Et comment généraliser la dérivée seconde?

Définition 1.5: Matrice jacobienne

Soit $f = (f_1, \dots, f_m): U \rightarrow \mathbb{R}^m$ différentiable sur un ouvert $U \subset \mathbb{R}^n$. La **matrice jacobienne** de f en $\mathbf{x} \in U$ est la matrice $m \times n$

$$J_f(\mathbf{x}) = \begin{pmatrix} \frac{\partial f_1}{\partial x_1}(\mathbf{x}) & \cdots & \frac{\partial f_1}{\partial x_n}(\mathbf{x}) \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1}(\mathbf{x}) & \cdots & \frac{\partial f_m}{\partial x_n}(\mathbf{x}) \end{pmatrix}.$$

Lorsque $m = 1$ (la fonction est scalaire), la Jacobienne est simplement le gradient transposé : $J_f(\mathbf{x}) = \nabla f(\mathbf{x})^\top$.

Définition 1.6: Matrice hessienne

Soit $f: U \rightarrow \mathbb{R}$ deux fois différentiable sur un ouvert $U \subset \mathbb{R}^n$. La **matrice hessienne** de f en $\mathbf{x} \in U$ est la matrice symétrique $n \times n$ des dérivées partielles secondes

$$\nabla^2 f(\mathbf{x}) = \left(\frac{\partial^2 f}{\partial x_i \partial x_j}(\mathbf{x}) \right)_{1 \leq i, j \leq n},$$

la symétrie découlant du théorème de Schwarz sous hypothèse de continuité des dérivées secondes.

Propriété 1.3: Développement de Taylor à l'ordre deux

Soit $f: U \rightarrow \mathbb{R}$ de classe C^2 au voisinage de $\mathbf{x} \in U$. Alors, pour $\mathbf{h} \in \mathbb{R}^n$ proche de $\mathbf{0}$,

$$f(\mathbf{x} + \mathbf{h}) = f(\mathbf{x}) + \nabla f(\mathbf{x})^\top \mathbf{h} + \frac{1}{2} \mathbf{h}^\top \nabla^2 f(\mathbf{x}) \mathbf{h} + o(\|\mathbf{h}\|^2).$$

Interprétation en Deep Learning : La formule de Taylor ci-dessus montre que la matrice Hessienne capture la **courbure** locale de la fonction de coût. Le gradient indique la pente, la Hessienne indique si cette pente s'aplatit (cuvette) ou s'accroît. Dans des espaces à millions de dimensions, la Hessienne aide à comprendre si l'on est dans un "minima plat" (bon pour la généralisation du modèle) ou un "minima aigu".

Exemple 1.4: Jacobienne et hessienne d'une fonction quadratique

Soit $f(\mathbf{x}) = \mathbf{x}^\top A \mathbf{x} + \mathbf{b}^\top \mathbf{x} + c$, avec $A \in \mathbb{R}^{n \times n}$ symétrique ($A^\top = A$), $\mathbf{b} \in \mathbb{R}^n$ et $c \in \mathbb{R}$. On obtient :

$$\nabla f(\mathbf{x}) = 2A\mathbf{x} + \mathbf{b}, \quad \nabla^2 f(\mathbf{x}) = 2A.$$

Démonstration :

Pour comprendre d'où viennent ces résultats, il faut réécrire $f(\mathbf{x})$ à l'aide de sommes scalaires (indices i et j). Soit x_i la i -ème composante du vecteur $\mathbf{x}$.

- Le terme constant c est un scalaire isolé.
- Le terme linéaire s'écrit : $\mathbf{b}^\top \mathbf{x} = \sum_{i=1}^n b_i x_i$.
- Le terme quadratique s'écrit : $\mathbf{x}^\top A \mathbf{x} = \sum_{i=1}^n \sum_{j=1}^n A_{i,j} x_i x_j$.

1. Calcul du Gradient $\nabla f(\mathbf{x})$

Calculons la dérivée partielle de f par rapport à une variable spécifique x_k (pour $k \in \{1, \dots, n\}$) :

$$\begin{aligned} \frac{\partial f}{\partial x_k} &= \frac{\partial}{\partial x_k} \left(\sum_{i=1}^n \sum_{j=1}^n A_{i,j} x_i x_j + \sum_{i=1}^n b_i x_i + c \right) \\ &= \sum_{i=1}^n \sum_{j=1}^n A_{i,j} \frac{\partial (x_i x_j)}{\partial x_k} + \sum_{i=1}^n b_i \frac{\partial x_i}{\partial x_k} + 0 \end{aligned}$$

Or, $\frac{\partial x_i}{\partial x_k}$ vaut 1 si $i = k$, et 0 sinon (symbole de Kronecker). Pour le terme linéaire :

$$\sum_{i=1}^n b_i \frac{\partial x_i}{\partial x_k} = b_k$$

Pour le terme quadratique, on applique la règle du produit $\frac{\partial (uv)}{\partial x} = u'v + uv'$:

$$\frac{\partial (x_i x_j)}{\partial x_k} = \left(\frac{\partial x_i}{\partial x_k} \right) x_j + x_i \left(\frac{\partial x_j}{\partial x_k} \right)$$

En injectant cela dans la double somme, on obtient deux termes :

$$\sum_{i=1}^n \sum_{j=1}^n A_{i,j} \left(\frac{\partial x_i}{\partial x_k} x_j + x_i \frac{\partial x_j}{\partial x_k} \right) = \sum_{j=1}^n A_{k,j} x_j + \sum_{i=1}^n A_{i,k} x_i$$

Le premier terme est la k -ème composante du vecteur $A\mathbf{x}$.

Le second terme est la k -ème composante du vecteur $A^\top \mathbf{x}$.

Puisque la matrice A est symétrique ($A = A^\top$), on a $A_{i,k} = A_{k,i}$. Donc les deux sommes sont identiques et valent $2 \sum_{j=1}^n A_{k,j} x_j$. En rassemblant le tout pour la composante k :

$$\frac{\partial f}{\partial x_k} = 2(A\mathbf{x})_k + b_k$$

En repassant sous forme vectorielle, on retrouve bien : $\nabla f(\mathbf{x}) = 2A\mathbf{x} + \mathbf{b}$.

2. Calcul de la Hessienne $\nabla^2 f(\mathbf{x})$

La matrice hessienne est la matrice des dérivées secondes. Soit la fonction composante

du gradient $g_k(\mathbf{x}) = \frac{\partial f}{\partial x_k} = 2 \sum_{j=1}^n A_{k,j} x_j + b_k$.

Dérivons g_k par rapport à une variable x_l :

$$\begin{aligned} \frac{\partial^2 f}{\partial x_k \partial x_l} &= \frac{\partial g_k}{\partial x_l} = \frac{\partial}{\partial x_l} \left(2 \sum_{j=1}^n A_{k,j} x_j + b_k \right) \\ &= 2 \sum_{j=1}^n A_{k,j} \frac{\partial x_j}{\partial x_l} + 0 \\ &= 2A_{k,l} \end{aligned}$$

Puisque l'élément à la position (k, l) de la matrice hessienne est exactement $2A_{k,l}$, on a sous forme matricielle : $\nabla^2 f(\mathbf{x}) = 2A$.

Interprétation en ML : Cette propriété est omniprésente. Dans une régression linéaire par moindres carrés, la fonction de coût est exactement de cette forme. La hessienne étant constante et égale à $2A$ (qui s'avère définie positive dans ce contexte), on a la preuve absolue que la surface d'erreur est un bol parfait (convexe), garantissant le succès de la descente de gradient vers un minimum global unique.

1.8 Applications à l'optimisation

1.8.1 Points critiques et condition nécessaire d'optimalité

L'objectif ultime est de trouver le "fond" de notre fonction de coût. Un fond est plat. Autrement dit, sa pente (son gradient) est nulle.

Définition 1.7: Point critique

Soit $f: U \rightarrow \mathbb{R}$ différentiable sur un ouvert $U \subset \mathbb{R}^n$. Un point $\mathbf{x}^* \in U$ est un **point critique** de f si $\nabla f(\mathbf{x}^*) = \mathbf{0}$.

Théorème 1.2: Condition nécessaire du premier ordre

Si $f: U \rightarrow \mathbb{R}$ est différentiable sur l'ouvert U et admet un extremum local (minimum ou maximum) en $\mathbf{x}^* \in U$, alors $\mathbf{x}^*$ est un point critique de f , c'est-à-dire $\nabla f(\mathbf{x}^*) = \mathbf{0}$.

Remarque 1.5: Condition nécessaire mais non suffisante

Un point critique peut être un minimum local, un maximum local, ou un point-selle (comme la forme d'une selle de cheval : un minimum dans une direction, un maximum dans une autre). Distinguer ces cas requiert une information d'ordre deux : les valeurs propres de la matrice hessienne (méthodes du second ordre, méthode de Newton).

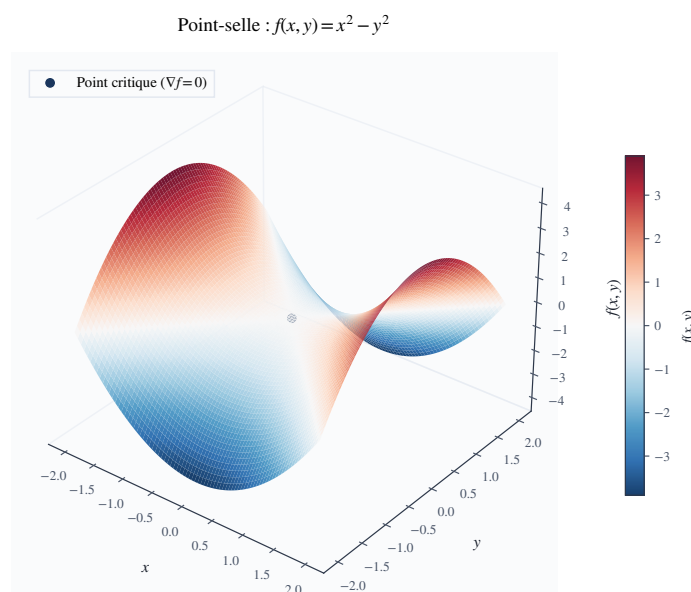


FIGURE 1.3 – Un point-selle (ou col) en $(0,0)$. Le gradient y est nul (le terrain est localement plat), mais ce n'est ni un minimum ni un maximum. L'analyse des valeurs propres de la matrice hessienne est indispensable pour identifier cette géométrie (ici, une valeur propre positive et une négative).

1.8.2 Convexité et caractérisation par la hessienne

En IA, nous aimons les fonctions *convexes*. Pourquoi ? Parce qu'un bol n'a qu'un seul fond. Tout point critique d'un bol est garanti d'être le minimum absolu.

Définition 1.8: Fonction convexe

Une fonction $f: \mathbb{R}^n \rightarrow \mathbb{R}$ est **convexe** si, pour tous $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ et tout $t \in [0, 1]$,

$$f(t\mathbf{x} + (1-t)\mathbf{y}) \leq tf(\mathbf{x}) + (1-t)f(\mathbf{y}).$$

Géométriquement, un segment reliant deux points de la courbe de f passe toujours au-dessus (ou sur) la courbe. Elle est **strictement convexe** si l'inégalité est stricte pour $\mathbf{x} \neq \mathbf{y}$ et $t \in (0, 1)$.

Théorème 1.3: Caractérisation de la convexité par la hessienne

Soit $f: \mathbb{R}^n \rightarrow \mathbb{R}$ de classe C^2 . Alors f est convexe si et seulement si $\nabla^2 f(\mathbf{x})$ est semi-définie positive pour tout $\mathbf{x} \in \mathbb{R}^n$. Si $\nabla^2 f(\mathbf{x})$ est définie positive pour tout $\mathbf{x}$, alors f est strictement convexe.

Propriété 1.4: Optimalité globale sous convexité

Si f est convexe et différentiable, alors tout point critique de f est un **minimum global**. Ce résultat justifie, en particulier, que pour une fonction de coût convexe (régression linéaire, régression logistique), un algorithme de descente de gradient convergeant vers un point critique converge vers l'optimum global. (Note : les réseaux de neurones profonds génèrent des fonctions de coût hautement **non-convexes**, d'où la complexité de leur optimisation).

1.8.3 Algorithme de descente de gradient

Voici enfin l'algorithme "roi" du Machine Learning moderne. C'est lui qui entraîne les modèles de génération de texte, la reconnaissance faciale, et les algorithmes de recommandation.

Algorithme 1 : Descente de gradient à pas constant

Entrées : fonction f différentiable (Coût), point initial $\mathbf{x}_0$ (Poids aléatoires initiaux), pas d'apprentissage (*learning rate*) $\alpha > 0$, tolérance $\varepsilon > 0$

Sorties : approximation $\mathbf{x}^*$ d'un point critique de f

```

1  $k \leftarrow 0$ ;
2 tant que  $\|\nabla f(\mathbf{x}_k)\| > \varepsilon$  faire
3    $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k)$ ; // Mise à jour dans le sens opposé au gradient
4    $k \leftarrow k + 1$ ;
5 fin
6 retourner  $\mathbf{x}_k$ 
```

Théorème 1.4: Convergence de la descente de gradient

Soit $f: \mathbb{R}^n \rightarrow \mathbb{R}$ différentiable, L -lisse (c'est-à-dire que ∇f est L -lipschitzienne) et bornée inférieurement. Si l'algorithme 1 est exécuté avec un pas constant $\alpha \in (0, 1/L]$, alors

$$\min_{0 \leq k \leq K-1} \|\nabla f(\mathbf{x}_k)\|^2 \leq \frac{2(f(\mathbf{x}_0) - f^*)}{\alpha K},$$

où $f^* = \inf_{\mathbf{x}} f(\mathbf{x})$. En particulier, le gradient tend vers 0 lorsque $K \rightarrow \infty$.

Démonstration. La L -lissité de f entraîne le lemme de descente

$$f(\mathbf{x}_{k+1}) \leq f(\mathbf{x}_k) - \alpha \|\nabla f(\mathbf{x}_k)\|^2 + \frac{L\alpha^2}{2} \|\nabla f(\mathbf{x}_k)\|^2.$$

Pour $\alpha \leq 1/L$, le coefficient de $\|\nabla f(\mathbf{x}_k)\|^2$ dans le membre de droite est majoré par $-\alpha/2$, d'où

$$f(\mathbf{x}_{k+1}) \leq f(\mathbf{x}_k) - \frac{\alpha}{2} \|\nabla f(\mathbf{x}_k)\|^2.$$

En sommant pour $k = 0, \dots, K - 1$ et en utilisant $f(\mathbf{x}_K) \geq f^*$, on obtient

$$\frac{\alpha}{2} \sum_{k=0}^{K-1} \|\nabla f(\mathbf{x}_k)\|^2 \leq f(\mathbf{x}_0) - f(\mathbf{x}_K) \leq f(\mathbf{x}_0) - f^*,$$

et il suffit de diviser par $\alpha K/2$ et de minorer la somme par K fois son plus petit terme. ■

Exemple 1.5: Descente de gradient sur une fonction quadratique

Pour $f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top A \mathbf{x}$ avec A symétrique définie positive de plus grande valeur propre L , on a $\nabla f(\mathbf{x}) = A \mathbf{x}$ et $\nabla^2 f(\mathbf{x}) = A$. Le choix du *learning rate* $\alpha = 1/L$ garantit la convergence de l'algorithme 1 vers l'unique minimum $\mathbf{x}^* = \mathbf{0}$, conformément au théorème 1.4.

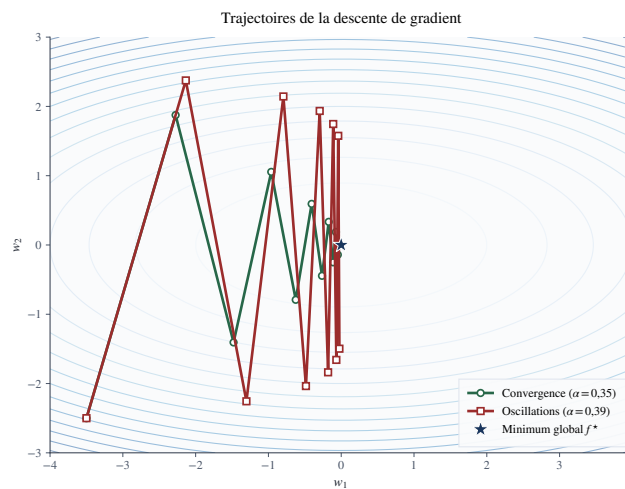


FIGURE 1.4 – Trajectoire des poids lors de la descente de gradient. Si le pas d'apprentissage α est bien calibré (vert), l'algorithme converge efficacement. S'il est trop grand (rouge), l'algorithme "rebondit" sur les parois de la vallée d'erreur (oscillations), ce qui ralentit considérablement la convergence, voire provoque une divergence.

Points clés du chapitre

Essentiels à retenir

- Un modèle d'IA et sa fonction de coût sont fondamentalement des **fonctions à plusieurs variables**.
- La dérivée partielle mesure la sensibilité locale d'une fonction par rapport à **un seul** paramètre, en figeant les autres.
- Le **gradient** rassemble ces dérivées partielles. Il indique la direction de plus forte croissance. **S'opposer** au gradient permet de minimiser l'erreur (Descente de gradient).
- La **jacobienne** généralise le gradient aux fonctions vectorielles; la **hessienne** encode l'information de courbure (dérivées secondes) et permet de diagnostiquer la nature d'un point critique (minimum, maximum, point-selle).
- La **règle de la chaîne** est le fondement mathématique de la **rétropropagation du gradient** (*backpropagation*), rendant possible l'entraînement de réseaux profonds.
- Un minimum local vérifie $\nabla f(\mathbf{x}^*) = \mathbf{0}$ (condition nécessaire); sous hypothèse de **convexité** de la fonction de coût, tout point critique est garanti d'être un minimum *global*.

Exercices et Travaux Pratiques (TP)

Partie I : Exercices Théoriques

1. Gradient de l'Erreur Quadratique (Régression Linéaire) :

En Machine Learning, l'erreur quadratique moyenne (MSE) pour un jeu de données de N points $\{(x_i, y_i)\}_{i=1}^N$ s'écrit pour un modèle linéaire $f(x) = wx + b$:

$$\mathcal{L}(w, b) = \frac{1}{2N} \sum_{i=1}^N (wx_i + b - y_i)^2$$

(a) Calculer les dérivées partielles $\frac{\partial \mathcal{L}}{\partial w}$ et $\frac{\partial \mathcal{L}}{\partial b}$.

(b) Montrer que si l'on pose $\nabla \mathcal{L}(w^*, b^*) = \mathbf{0}$, on retrouve les formules statistiques classiques de la covariance et de la variance pour w^* .

2. Calcul complet et optimisation analytique :

Soit la fonction $f(x, y) = x^2 e^y - 3xy^2$.

(a) Calculer le gradient $\nabla f(x, y)$.

(b) Calculer la matrice hessienne $\nabla^2 f(x, y)$.

(c) Évaluer le gradient et la hessienne au point $A = (1, 0)$. Dans quelle direction la fonction décroît-elle le plus vite depuis A ?

3. Distance, Gradient et Convexité :

Soit $g(\mathbf{x}) = \|\mathbf{x} - \mathbf{a}\|^2$ pour $\mathbf{a} \in \mathbb{R}^n$ fixé.

(a) Montrer, en développant la norme ou en utilisant la définition du gradient matriciel, que $\nabla g(\mathbf{x}) = 2(\mathbf{x} - \mathbf{a})$.

(b) En déduire l'unique point critique de g .

(c) Calculer la hessienne de g . Que peut-on en conclure sur la nature du point critique (minimum, maximum ou point-selle)?

4. Les limites du critère de la hessienne :

Montrer que la fonction $h(x) = x^4$ est strictement convexe sur $\mathbb{R}$ mais que sa dérivée seconde (hessienne en 1D) s'annule en $x = 0$. Que peut-on en conclure sur la réciproque du théorème 1.3?

Partie II : Travaux Pratiques sous Python (NumPy, SymPy, Matplotlib)

Ce TP a pour objectif de coder "from scratch" (sans bibliothèques de Machine Learning comme Scikit-Learn ou PyTorch) l'algorithme d'entraînement fondamental de l'IA : la Descente de Gradient.

5. Vérification symbolique avec SymPy :

Dans un *Notebook Jupyter*, importez `sympy`.

(a) Déclarez vos variables symboliques `x, y = sp.symbols('x y')`.

(b) Implémentez la fonction de l'exercice 2 : $f(x, y) = x^2 e^y - 3xy^2$.

(c) Utilisez `sp.diff()` pour calculer le gradient et `sp.hessian()` pour calculer la matrice hessienne. Comparez les résultats affichés avec vos calculs manuels.

6. Génération de données "Jouets" avec NumPy :

Simulons un jeu de données réel.

- (a) Générez un vecteur X de 100 valeurs aléatoires uniformes entre 0 et 10.
- (b) Générez un vecteur cible y tel que $y = 3x + 2 + \varepsilon$, où ε est un bruit gaussien $\mathcal{N}(0, 1)$ généré via `np.random.randn`.
- (c) Tracez le nuage de points avec `plt.scatter`.

7. Implémentation de la Descente de Gradient :

À partir de l'Algorithme 1 du cours :

- (a) Créez une fonction `compute_loss(w, b, X, y)` qui retourne la MSE.
- (b) Créez une fonction `compute_gradients(w, b, X, y)` qui retourne un tuple (dw, db) en utilisant les formules mathématiques trouvées à l'exercice 1.
- (c) Codez la boucle d'entraînement `gradient_descent(X, y, w_init, b_init, alpha, epochs)` qui met à jour w et b . Stockez la valeur de la perte (*loss*) à chaque itération dans une liste.

8. Visualisation de l'apprentissage (Courbe de perte) :

Exécutez votre algorithme avec $w_{init} = 0$, $b_{init} = 0$, un *learning rate* $\alpha = 0.01$ et 100 époques.

- (a) Tracez l'évolution de la fonction de perte en fonction du numéro de l'époque. Que remarquez-vous sur la forme de la courbe ?
- (b) Affichez la droite de régression obtenue $y_{pred} = w^*X + b^*$ sur le même graphique que le nuage de points de la question 6.

9. Exploration de la surface de coût et divergence (Hyperparamètres) :

Le choix du pas α (taux d'apprentissage) dépend directement de la constante de Lipschitz (liée à la courbure/Hessienne).

- (a) À l'aide de `plt.contour` ou `plt.contourf`, tracez les lignes de niveau de la fonction de coût $\mathcal{L}(w, b)$ dans l'espace des paramètres (faites varier $w \in [0, 5]$ et $b \in [0, 4]$).
- (b) Superposez sur ce graphique la trajectoire de vos poids (w_k, b_k) à chaque itération. Observez comment l'algorithme "descend" dans la vallée perpendiculairement aux lignes de niveau.
- (c) **Crash test** : Relancez l'entraînement avec un pas d'apprentissage trop grand (ex : $\alpha = 0.5$). Tracez à nouveau la trajectoire. Que se passe-t-il ? Reliez cette observation mathématiquement au Théorème 1.4.

Chapitre 2

Algèbre linéaire

Introduction générale

Si le calcul différentiel (Chapitre 1) indique *comment* ajuster un modèle, l'algèbre linéaire fournit le *langage* dans lequel les paramètres et les données sont représentés. L'apprentissage profond (*Deep Learning*) n'est fondamentalement qu'une succession de transformations géométriques fluides dans des espaces à très haute dimension. Pour l'ordinateur, l'image est une matrice et le texte un tenseur ; ignorer leur structure interne (rang, conditionnement, spectre) mène inévitablement à des modèles instables (les fameux *NaN* lors de l'entraînement) ou inefficaces en mémoire.

Ce chapitre pose rigoureusement ces fondations matricielles. Nous y définirons les **tenseurs** (structures reines de l'IA) et le *broadcasting*, avant de décortiquer le produit matriciel comme une composition géométrique. La notion de **rang** y sera directement reliée à la capacité de mémorisation d'un modèle (expliquant l'approche moderne *LoRA*).

Enfin, après avoir résolu la régression linéaire via les équations normales, nous insisterons sur l'impact vital du **conditionnement matriciel** sur l'optimisation. L'étude des **valeurs et vecteurs propres** (qui expliquent l'explosion des gradients dans les RNN) nous mènera à la Décomposition en Valeurs Singulières (**SVD**) : bien plus qu'un théorème élégant, c'est l'algorithme originel d'extraction de ce que l'IA appelle un « Espace Latent ».

Objectif

Objectif : Maîtriser les fondations géométriques et matricielles avancées du Machine Learning pour concevoir des architectures stables, comprendre la réduction de dimension, et penser en termes de *tenseurs* plutôt que de *boucles itératives*.

Masse horaire : 4 h de cours • 3 h de TD/TP.

• Outils & bibliothèques

NumPy, SciPy, Matplotlib

2.1 Scalaires, vecteurs, matrices, tenseurs et dimensionnalité

Dans les langages de programmation classiques, on traite les données séquentiellement via des boucles `for` ou `while`. En Intelligence Artificielle, cette approche est proscrite car elle ne permet pas d'exploiter les milliers de cœurs de calcul parallèles d'une carte graphique (GPU). L'unité fondamentale de calcul n'est donc pas le nombre isolé, mais le **tenseur**.

Définition 2.1: scalaire, vecteur, matrice, tenseur

En algèbre multilinéaire, on classe les objets selon leur nombre d'axes (ou dimensions, ou *modes*) :

- **Scalaire (Ordre 0)** : Un simple nombre $a \in \mathbb{R}$. Il n'a aucun axe.
- **Vecteur (Ordre 1)** : Un tableau à un seul axe, $\mathbf{v} = (v_1, \dots, v_n)^\top \in \mathbb{R}^n$. Par convention stricte en algèbre, un vecteur est toujours un vecteur *colonne*.
- **Matrice (Ordre 2)** : Un tableau à deux axes (lignes et colonnes), $A \in \mathbb{R}^{m \times n}$, de coefficients A_{ij} (ligne i , colonne j).
- **Tenseur (Ordre $k \geq 3$)** : Un hyper-tableau multidimensionnel à k axes, $T \in \mathbb{R}^{n_1 \times n_2 \times \dots \times n_k}$.

Remarque 2.1: Anatomie d'un tenseur de données en Deep Learning

La forme (appelée *shape*) d'un tenseur dicte son organisation en mémoire. Prenons l'exemple d'un lot (*batch*) de 32 images couleurs en Haute Définition (1920×1080). Dans un framework comme PyTorch, ces données sont regroupées dans un tenseur d'ordre 4 de *shape* (32, 3, 1080, 1920) :

1. L'axe 0 est la **taille du batch** (32 images).
2. L'axe 1 est le nombre de **canaux de couleur** (3 pour Rouge, Vert, Bleu).
3. L'axe 2 est la **hauteur** spatiale (1080 pixels).
4. L'axe 3 est la **largeur** spatiale (1920 pixels).

Propriété 2.1: Broadcasting (Diffusion Numérique)

Le *broadcasting* est le mécanisme fondamental qui permet d'effectuer des opérations mathématiques entre des tenseurs de tailles différentes sans avoir à dupliquer explicitement les données en mémoire. Si l'on veut ajouter un vecteur de biais $\mathbf{b} \in \mathbb{R}^{1 \times n}$ à une matrice de données $X \in \mathbb{R}^{m \times n}$:

$$(X + \mathbf{b})_{ij} = X_{ij} + b_j$$

Le vecteur $\mathbf{b}$ est virtuellement "étiré" (diffusé) le long de l'axe des lignes (axe 0) pour correspondre à la taille m de la matrice X .

Bon à savoir : La convention de sommation d'Einstein (einsum)

Dans les architectures modernes (comme les Transformers à la base de ChatGPT), manipuler des tenseurs d'ordre 4 ou 5 devient un cauchemar syntaxique avec les produits matriciels classiques. L'opération einsum spécifie explicitement les axes à contracter via des chaînes de caractères.

- **Produit scalaire** $\mathbf{u}^\top \mathbf{v} : i, i \rightarrow$ (on somme sur l'axe i , le résultat n'a pas d'axe, c'est un scalaire).
- **Produit matriciel** $C = AB : ik, kj \rightarrow ij$ (on somme sur la dimension commune k , on garde i et j).
- **Mécanisme d'Attention (Multi-Head)** : Le calcul des scores d'attention requiert de multiplier un tenseur de Requêtes (Queries Q) et de Clés (Keys K).
Leur *shape* est (batch, heads, seq_len, d_model).
En einsum, cela s'écrit en une ligne de code sublime : `bhqd, bhkd -> bhqk`. On effectue un produit matriciel sur les dimensions de séquence (q, k) et de modèle (d), tout en appliquant cela indépendamment pour chaque exemple du batch (b) et chaque head (h).

2.2 Produit matriciel, Trace, Rang et Compression Sémantique

Définition 2.2: Produit matriciel comme Composition géométrique

Le produit matriciel $C = AB$ est défini analytiquement par $C_{ij} = \sum_k A_{ik}B_{kj}$. Cependant, géométriquement, si $B: \mathbb{R}^p \rightarrow \mathbb{R}^n$ et $A: \mathbb{R}^n \rightarrow \mathbb{R}^m$ sont des applications linéaires, alors la matrice AB représente l'**application composée** $A \circ B$, qui projette directement de $\mathbb{R}^p$ vers $\mathbb{R}^m$.

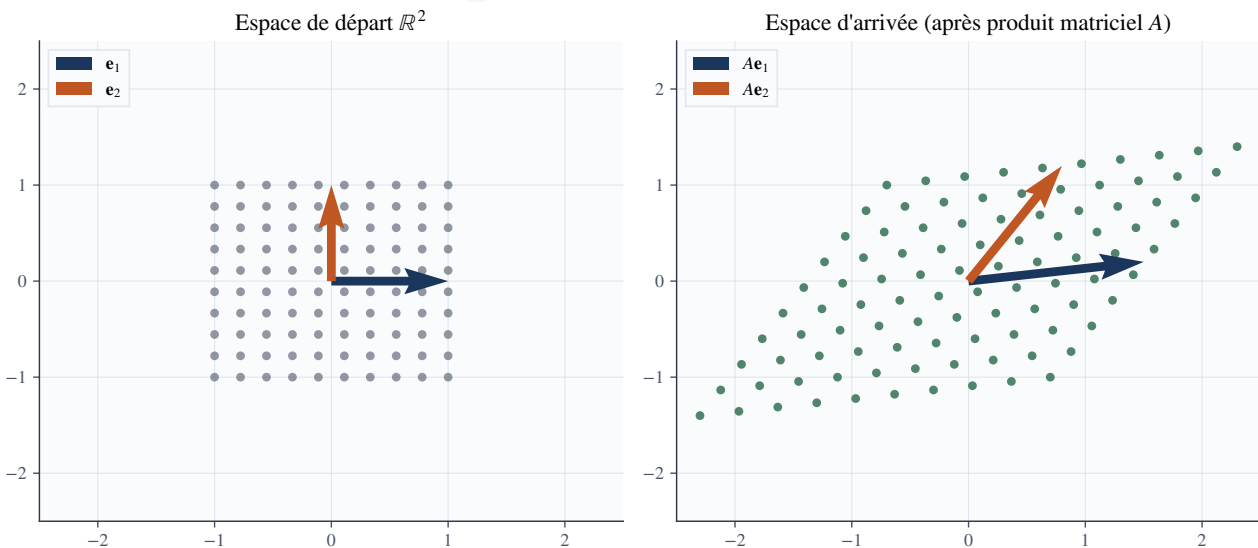


FIGURE 2.1 – Interprétation géométrique du produit matriciel. Une matrice $A \in \mathbb{R}^{2 \times 2}$ appliquée à une grille de points dans $\mathbb{R}^2$ agit comme une déformation de l'espace (ici une combinaison de rotation et d'étirement). L'aire d'un carré unitaire formé par les vecteurs de base devient un parallélogramme dont la surface est exactement égale au déterminant de A .

Exemple 2.1: Le calcul d'une couche de réseau de neurones en détail

Prenons une couche de réseau Dense (*Fully Connected*) recevant un lot de $N = 32$ observations, chacune contenant $d = 784$ caractéristiques (ex : pixels). L'entrée est la matrice $X \in \mathbb{R}^{32 \times 784}$. La couche contient $h = 128$ neurones. La matrice des poids est définie telle que chaque ligne représente un neurone : $W \in \mathbb{R}^{128 \times 784}$. Le vecteur de biais est $\mathbf{b} \in \mathbb{R}^{128}$. La transformation de toute la couche s'écrit en un seul produit matriciel :

$$Y = XW^T + \mathbf{b}$$

Analyse des dimensions :

1. W^T a pour dimension (784×128) .
2. Le produit $X \times W^T$ oppose une matrice (32×784) à une matrice (784×128) . L'axe intérieur (784) disparaît par sommation. Le résultat est une matrice (32×128) .
3. Le vecteur $\mathbf{b}$ de dimension (128) est ajouté à chaque ligne de la matrice résultante grâce au *broadcasting*.

Conclusion : Nous avons transformé 32 vecteurs de dimension 784 en 32 vecteurs de dimension 128 en une seule instruction optimisée pour le GPU.

Propriété 2.2: Transposée d'un produit

$(AB)^T = B^T A^T$. *Interprétation* : L'ordre s'inverse. En Deep Learning, lors de la passe avant (*forward*), l'information circule selon $x \mapsto Wx$. Lors de la passe arrière (*rétropropagation* du Chapitre 1), le gradient d'erreur δ remonte le graphe de calcul dans le sens opposé via la transposée : $\delta \mapsto W^T \delta$.

Définition 2.3: Rang et Indépendance Linéaire

Le **rang** d'une matrice $A \in \mathbb{R}^{m \times n}$, noté $\text{rg}(A)$, est le nombre maximal de colonnes (ou de lignes) qui sont **linéairement indépendantes**. Par exemple, si la colonne 3 est exactement égale à la somme de la colonne 1 et 2, elle n'apporte aucune information géométrique nouvelle. Le rang est la dimension réelle de l'espace généré (l'image de A). On a nécessairement $\text{rg}(A) \leq \min(m, n)$.

Interprétation ML : Effondrement Dimensionnel et LoRA

Considérez une matrice de poids d'un réseau immense $W \in \mathbb{R}^{10000 \times 10000}$. Elle contient 100 millions de paramètres. Si $\text{rg}(W) = 8$, cela signifie que malgré sa taille gigantesque, cette matrice écrase toute donnée d'entrée sur un sous-espace de seulement 8 dimensions ! Il y a énormément de redondance.

Cette observation est la base de **LoRA (Low-Rank Adaptation)**. Pour adapter un modèle LLM massif comme LLaMA sans réentraîner tous ses poids initiaux W_0 , on gèle W_0 et on apprend une petite perturbation ΔW . Au lieu d'apprendre 100 millions de valeurs, on force ΔW à être de rang faible en la décomposant : $\Delta W = AB$, où $A \in \mathbb{R}^{10000 \times 8}$ et $B \in \mathbb{R}^{8 \times 10000}$. Le nombre de paramètres à optimiser chute à $2 \times (10000 \times 8) = 160\,000$. **On vient de réduire la charge d'entraînement de 99.8 %**, rendant possible l'affinage de l'IA sur un simple ordinateur personnel.

2.3 Inversion Matricielle, Équations Normales et Conditionnement

Définition 2.4: Déterminant et Inverse

Le **déterminant** $\det(A)$ d'une matrice carrée mesure le facteur de dilatation du volume (ou aire) d'un hypercube après application de la transformation A . Si $\det(A) = 0$, le volume s'écrase totalement sur une dimension inférieure (le rang n'est pas plein). La matrice n'est pas **inversible** car on ne peut pas reconstruire un volume 3D à partir d'une ombre 2D (l'information perdue est irrécupérable).

Exemple 2.2: Régression Linéaire (Équations Normales)

L'algèbre linéaire permet de trouver le minimum absolu d'un modèle linéaire sans même utiliser la descente de gradient. Soit $X \in \mathbb{R}^{N \times d}$ la matrice de données et $y \in \mathbb{R}^N$ les cibles. La fonction de coût d'Erreur Quadratique (MSE) s'écrit sous forme vectorielle :

$$\mathcal{L}(w) = \|Xw - y\|_2^2 = (Xw - y)^\top (Xw - y)$$

Développons cette expression :

$$\mathcal{L}(w) = (w^\top X^\top - y^\top)(Xw - y) = w^\top X^\top Xw - w^\top X^\top y - y^\top Xw + y^\top y$$

Puisque le résultat est un scalaire, sa transposée est égale à lui-même : $w^\top X^\top y = (w^\top X^\top y)^\top = y^\top Xw$. On simplifie :

$$\mathcal{L}(w) = w^\top (X^\top X)w - 2w^\top X^\top y + y^\top y$$

Pour trouver le minimum, on calcule le gradient par rapport au vecteur w et on l'égalise au vecteur nul $\mathbf{0}$ (condition du 1^{er} ordre du Chapitre 1). En utilisant les identités de calcul matriciel $\nabla_w(w^\top Aw) = 2Aw$ (si A est symétrique) et $\nabla_w(w^\top b) = b$:

$$\nabla_w \mathcal{L}(w) = 2(X^\top X)w - 2X^\top y = \mathbf{0}$$

Ce qui nous donne directement les **Équations Normales** :

$$(X^\top X)w = X^\top y \implies w^* = (X^\top X)^{-1} X^\top y$$

Cependant, en Machine Learning, une matrice n'est presque jamais *exactement* non-inversible, mais elle peut être *presque* non-inversible. C'est ici qu'intervient le conditionnement.

Définition 2.5: Nombre de Conditionnement (κ)

Le **nombre de conditionnement** d'une matrice symétrique définie positive A (comme $X^\top X$ ou la Hessienne $\nabla^2 \mathcal{L}$) est le rapport entre sa plus grande et sa plus petite valeur propre :

$$\kappa(A) = \frac{\lambda_{\max}}{\lambda_{\min}}$$

Remarque 2.2: Le Ravin et l'Optimisation

Le conditionnement κ mesure la "rotondité" de la surface de coût. Si $\kappa \approx 1$ ($\lambda_{\max} \approx \lambda_{\min}$), la surface d'erreur est un bol parfait. La descente de gradient converge en ligne droite vers le centre en très peu d'itérations. Si $\kappa = 1000$ (une valeur propre est 1000 fois plus grande que l'autre), la surface est un ravin extrêmement allongé. Le gradient rebondira violemment sur les parois abruptes du ravin tout en avançant très lentement au fond. C'est la raison majeure pour laquelle on normalise toujours les données (StandardScaler) avant d'entraîner une IA : pour forcer le conditionnement de la matrice vers 1.

2.4 Valeurs propres, Vecteurs propres et Dynamique des RNN

Définition 2.6: Spectre d'une matrice

Un scalaire $\lambda \in \mathbb{C}$ est une **valeur propre** et $\mathbf{v} \neq \mathbf{0}$ son **vecteur propre** associé si la matrice A agit sur $\mathbf{v}$ de façon purement scalaire :

$$A\mathbf{v} = \lambda\mathbf{v}.$$

L'ensemble des valeurs propres est appelé le **spectre** de A . L'axe défini par $\mathbf{v}$ est invariant : la matrice ne fait qu'étirer ou contracter l'espace selon cet axe, avec une force λ .

Théorème 2.1: Théorème spectral (Le cœur de l'ACP)

Toute matrice **symétrique** $A \in \mathbb{R}^{n \times n}$ (comme une matrice de covariance) est diagonalisable dans une base orthonormée (ses vecteurs propres sont perpendiculaires entre eux). Il existe une matrice orthogonale Q ($Q^\top Q = I_n$) et une matrice diagonale Λ telles que :

$$A = Q\Lambda Q^\top$$

Toutes ses valeurs propres sont réelles. L'Analyse en Composantes Principales (ACP) utilise précisément Q pour projeter les données sur les axes de plus forte variance (plus fortes valeurs propres).

Exemple 2.3: Diagonalisation détaillée ligne par ligne

Prenons $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$. 1. **Valeurs propres** : Racines de $\det(A - \lambda I) = 0$.

$$\det \begin{pmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{pmatrix} = (2-\lambda)^2 - 1 = \lambda^2 - 4\lambda + 3 = 0.$$

Les solutions sont $\lambda_1 = 3$ et $\lambda_2 = 1$.

2. **Vecteurs propres** : Résoudre $A\mathbf{v} = \lambda\mathbf{v}$.

Pour $\lambda_1 = 3$: $\begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = 3 \begin{pmatrix} x \\ y \end{pmatrix} \implies 2x + y = 3x \implies y = x$. Le vecteur est

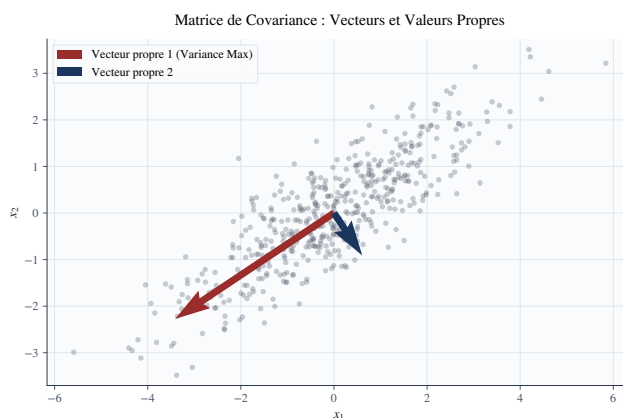


FIGURE 2.2 – Application du Théorème Spectral à une matrice de covariance. Le nuage de points génère une matrice symétrique définie positive. La diagonalisation révèle les axes naturels de la distribution : les vecteurs propres. La longueur de l’axe principal (en rouge) est proportionnelle à la racine de la valeur propre associée λ_1 , justifiant mathématiquement le principe de l’Analyse en Composantes Principales (ACP).

$$\mathbf{u}_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Pour $\lambda_2 = 1 : 2x + y = x \implies y = -x$. Le vecteur est $\mathbf{u}_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$.

On remarque que $\mathbf{u}_1^\top \mathbf{u}_2 = \frac{1}{2}(1 - 1) = 0$. Ils sont bien orthogonaux !

Bon à savoir : Le Rayon Spectral et la mort des RNN

Dans un Réseau de Neurones Récurrent (RNN) utilisé pour les séries temporelles ou le texte, l’état caché h_t se met à jour selon une matrice de poids $W : h_t = Wh_{t-1}$. Déplier la récurrence sur T pas de temps donne $h_T = W^T h_0$.

Si on diagonalise $W = Q\Lambda Q^{-1}$, alors $W^T = Q\Lambda^T Q^{-1}$.

La limite quand $T \rightarrow \infty$ dépend de la plus grande valeur propre en valeur absolue, appelée **Rayon Spectral** $\rho(W)$:

- Si $\rho(W) > 1$, la valeur propre à la puissance T explose vers l’infini. C’est le problème de l’**Explosion du Gradient** (le réseau devient instable).
- Si $\rho(W) < 1$, $\lambda^T \rightarrow 0$. C’est le **Vanishing Gradient**. Le réseau oublie totalement le passé lointain (amnésie).

L’algèbre linéaire démontre ici pourquoi l’architecture RNN classique est défectueuse, justifiant la création des LSTMs ou des Transformers.

2.5 SVD : Décomposition en Valeurs Singulières et Espaces Latents

Le théorème spectral est magique, mais ne s’applique qu’aux matrices carrées symétriques. Que faire avec une matrice de données $X \in \mathbb{R}^{m \times n}$ quelconque (ex : Utilisateurs $\times$ Produits) ? La réponse est la SVD.

Théorème 2.2: Décomposition en valeurs singulières (SVD)

Toute matrice $X \in \mathbb{R}^{m \times n}$ (de rang r) admet une factorisation universelle :

$$X = U\Sigma V^\top$$

- $U \in \mathbb{R}^{m \times m}$ est une matrice orthogonale. Ses colonnes, les vecteurs singuliers gauches, décrivent la structure des lignes (ex : profils des utilisateurs).
- $V \in \mathbb{R}^{n \times n}$ est une matrice orthogonale. Ses colonnes, les vecteurs singuliers droits, décrivent la structure des colonnes (ex : caractéristiques des produits).
- $\Sigma \in \mathbb{R}^{m \times n}$ est une matrice diagonale contenant les **valeurs singulières** $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$. Elles représentent la "force" ou "l'importance" du signal capturé.

Théorème 2.3: Théorème d'Eckart-Young-Mirsky

Si on met à zéro toutes les valeurs singulières de Σ sauf les k premières, on obtient une matrice tronquée $\tilde{\Sigma}_k$. La matrice reconstituée :

$$X_k = U\tilde{\Sigma}_k V^\top = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$$

est la **meilleure approximation mathématique absolue** (Compression Optimale) de rang k de la matrice originale X (minimisant l'erreur quadratique $\|X - X_k\|_F^2$).

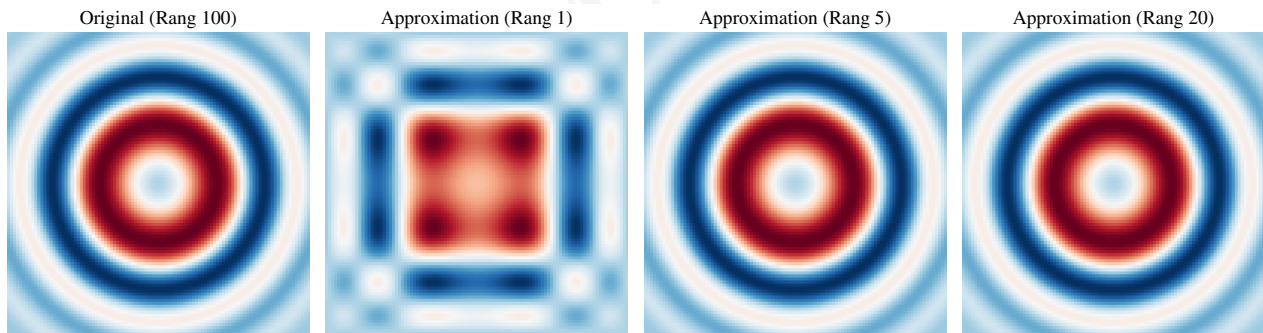


FIGURE 2.3 – Illustration du Théorème d'Eckart-Young via la Décomposition en Valeurs Singulières (SVD). Une matrice (ici représentée par une carte de chaleur de 100×100) de rang plein peut être approchée avec une fidélité étonnante en ne conservant qu'une fraction de ses valeurs singulières. Avec un rang $k = 20$, 99% de la variance structurelle est préservée pour un stockage drastiquement réduit.

L'essence de l'IA (Les Espaces Latents) :

Dans un système de recommandation de films (Netflix), X est une matrice immense, creuse et bruitée. En effectuant une SVD et en ne gardant que $k = 50$ composantes, on force les données à passer dans un **espace latent** restreint. La matrice ne décrit plus les films par leurs pixels ou leur titre, mais par 50 "concepts latents" cachés découverts par la machine (ex : le vecteur $\mathbf{u}_1$ pourrait représenter "l'affinité pour les films d'action des années 80"). C'est le fondement de la compréhension sémantique de l'IA.

Points clés du chapitre

Essentiels à retenir

- L'IA moderne requiert de **vectoriser** les opérations sous forme tensorielle pour saturer l'architecture des GPUs. Le *broadcasting* et *einsum* sont les outils syntaxiques de cette révolution.
- Le **rang** d'une matrice mesure la vraie quantité d'information linéairement indépendante qu'elle contient. Les matrices de rang faible (Low-Rank) permettent de compresser drastiquement les réseaux de neurones gigantesques.
- Le **conditionnement matriciel** régit la topologie du paysage d'erreur. Un mauvais conditionnement crée un "ravin" qui paralyse la descente de gradient classique.
- Le **rayon spectral** dicte la stabilité asymptotique des systèmes récurrents (RNN). Si $\rho \neq 1$, les gradients s'évanouissent ou explosent.
- La **SVD** ($X = U\Sigma V^T$) est la technique matricielle suprême. Couplée au théorème d'Eckart-Young, elle permet le débruitage, la compression, et surtout la projection des données dans des **espaces sémantiques latents**.

Exercices et Travaux Pratiques (TP)

Partie I : Exercices Théoriques (Calcul Matriciel & ML)

1. Inversion d'un bloc et stabilité :

Prouvez, en utilisant les propriétés des déterminants, que si la matrice des données X possède deux colonnes parfaitement colinéaires, alors le déterminant de la Hessienne de la régression linéaire ($X^T X$) est rigoureusement nul.

2. Semi-Définie Positivité et Convexité globale :

Soit la matrice de Gram $M = X^T X$.

- Montrez que pour tout vecteur $z \neq \mathbf{0}$, $z^T M z \geq 0$. (Indice : introduire le vecteur $v = Xz$ et utiliser la norme L_2).
- Sachant que la Hessienne de la fonction de coût de régression est proportionnelle à $X^T X$, que concluez-vous sur la topologie de la surface (Convexité) d'après le Chapitre 1 ?

3. Pseudo-Inverse de Moore-Penrose :

La SVD d'une matrice X rectangulaire est $X = U \Sigma V^T$. On définit la pseudo-inverse par $X^+ = V \Sigma^+ U^T$ (où Σ^+ s'obtient en inversant les éléments non nuls de Σ et en transposant la matrice). Prouvez la condition géométrique de Penrose : $XX^+X = X$.

Partie II : Travaux Pratiques sous Python (Du Ravin au Transformer)

Ce TP "from scratch" sous Python aborde les concepts les plus avancés de l'algèbre linéaire en IA, sans utiliser de frameworks de haut niveau.

4. Visualisation de la malédiction du Conditionnement (Le "Ravin") :

Considérez une fonction quadratique de coût $f(x, y) = \frac{1}{2}(ax^2 + by^2)$. Sa matrice Hessienne est diagonale avec pour valeurs propres a et b .

- Le Bol Parfait** : Fixez $a = 1, b = 1$ (Conditionnement $\kappa = 1$). Tracez les lignes de niveau (`plt.contour`). Implémentez une simple descente de gradient et superposez la trajectoire des poids. Observez la convergence rectiligne.
- Le Ravin** : Fixez $a = 1, b = 20$ (Conditionnement $\kappa = 20$). Tracez les lignes de niveau (des ellipses étirées). Relancez l'optimisation. Visualisez les oscillations destructrices (zig-zag). Expliquez pourquoi un grand rapport entre les valeurs propres ruine la dynamique d'apprentissage.

5. Implémentation de l'algorithme "Power Iteration" (SVD sous le capot) :

Derrière l'appel `np.linalg.svd`, se cache un algorithme itératif redoutable pour trouver la plus grande valeur singulière.

- Initialisez une matrice X aléatoire 50×30 et un vecteur unitaire aléatoire $\mathbf{v} \in \mathbb{R}^{30}$.
- Répétez 100 fois la boucle de projection suivante : projetez $\mathbf{u} = X\mathbf{v}$, puis mettez à jour $\mathbf{v} = X^T \mathbf{u}$. À chaque étape, normalisez $\mathbf{v} = \mathbf{v} / \|\mathbf{v}\|_2$.
- À la fin de la boucle, calculez $\sigma_1 = \|X\mathbf{v}\|_2$.
- Comparez ce scalaire avec la valeur renvoyée par `np.linalg.svd(X)[1][0]`. Conclusion ? Vous venez de coder la méthode mathématique à la base de l'algorithme de classement *PageRank* de Google.

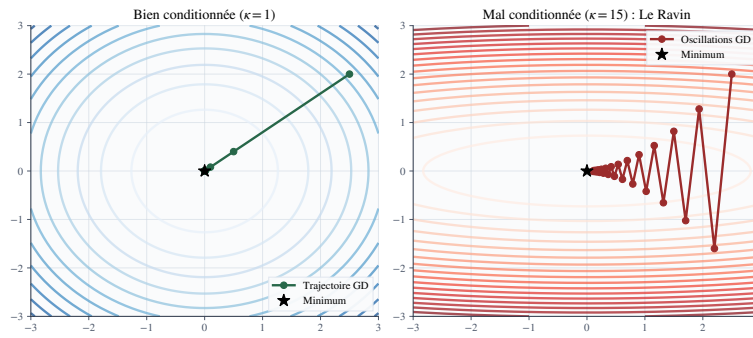


FIGURE 2.4 – Impact du conditionnement matriciel κ sur l'optimisation. À gauche : la matrice Hessienne est l'identité ($\kappa = 1$), les lignes de niveau sont parfaitement circulaires et l'algorithme converge en ligne droite. À droite : $\kappa = 15$ crée un "ravin". Le gradient (qui est orthogonal aux lignes de niveau elliptiques) pointe presque exclusivement vers les murs et très peu vers le minimum. L'algorithme rebondit violemment.

6. LE SUMMUM : Implémentation du mécanisme de Self-Attention :

L'architecture Transformer (Ex : ChatGPT) repose intégralement sur une seule équation d'algèbre linéaire :

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V$$

Soit une phrase de $N = 4$ mots, représentée par une matrice d'embeddings $X \in \mathbb{R}^{4 \times 3}$.

- Définissez X avec `np.random.randn`. Créez trois matrices de poids synaptiques arbitraires $W_Q, W_K, W_V \in \mathbb{R}^{3 \times 3}$.
- Calculez les projections linéaires : Les Requêtes ($Q = XW_Q$), Les Clés ($K = XW_K$), et Les Valeurs ($V = XW_V$).
- Calculez la matrice des scores bruts $S = QK^\top$. Quelle est sa dimension ? Que représente l'élément scalaire $S_{i,j}$ géométriquement entre le mot i et le mot j (pensez au produit scalaire) ?
- Divisez S par $\sqrt{3}$, puis appliquez la fonction Softmax par ligne (`scipy.special.softmax(S, axis=1)`). Vous obtenez la matrice d'attention A . Prouvez que la somme de chaque ligne vaut 1.
- Calculez enfin la sortie du module : $Z = AV$.

Analyse : Comparez la ligne 1 de Z avec la matrice V . Montrez comment l'algèbre linéaire permet au vecteur de sortie du mot 1 d'être une "combinaison linéaire convexe" des vecteurs de valeurs de *tous* les autres mots, créant ainsi le **contexte**.